

Github Link (tba)

[Github Link](#)

Rapport for Øving_3 AlgData (Alternativ 1)

Alternativet som ble valgt i denne oppgaven var å sammenligne to varianter av en sorteringsalgoritme, nemlig quicksort. Vi sammenligner en **quicksort med ett delingstall (refereres som Quicksort i resten av dokumentet)** og en quicksort som bruker to, nemlig **Dual Pivot quicksort**.

Quicksort algoritmen som brukes er den funnet i boka og bruker hjelpe funksjonene funnet i kapitell 3.9.1, 3.9.2 og 3.9.3. Fra boka får vi vite at gjennomsnittlige kjøretiden er $O(n \log(n))$ og $O(n^2)$ i værste.

Dual Pivot Quicksort algoritmen er en kopi og intregret fra [GeeksforGeeks](#), og vil ha forbedringene sagt i oppgavebeskrivelsen; Bytting av tallet i `arr[low]` med `arr[low + (high - low)/3]` og tilsvarende, og ved duplicater gjøres det ikke en rekursiv kall. I nettsiden får vite at kjøretiden ligger i $O(\log n)$ og $O(n)$ i noen tilfeller. Ifølge nettsiden, er den værste tilfellet når det lista er allerede sortert, enten økende eller minkende, der er tiden på $O(n^2)$.

Koden

Hjelpe funksjoner og Assert-funksjoner

For å bytte to verdier sine posisjoner brukes denne funksjonen:

```
void bytt(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}
```

For å sjekke om algoritmen ikke endrer selve verdiene, sjekker vi immutabilitet gjennom å sjekke summen. For å sjekke om det er faktisk sortert riktig, sammenligner vi fra bunn til topp om det er stigende;

```
int sum(int list[], int length)
{
    int sum = 0;
    for (int i = 0; i < length; i++)
    {
        sum += list[i];
    }
    return sum;
}
```

```
// A sortert liste er en stigende list. F.eks. {2, 6, 9, 11, 21, 67}
void assertSorted(int list[], int length)
{
    if (length < 2)
    {
        printf("List too short\n");
        return;
    }

    for (int i = 1; i < length; i++)
    {
        if (list[i] < list[i - 1])
        {
            fprintf(stderr, "Assertion failed: list[%d] (%d) < list[%d] (%d)\n",
i, list[i], i - 1, list[i - 1]);
            return;
        }
    }

    printf("List is sorted\n");
}
```

Disse funksjonene lager nye lister, ene lager en liste med tilfeldige tall og den andre en liste der annenhver tall er like.

```
// Funksjon som returnerer tilfeldig array:
int *createRandomArray(int count, int min, int max)
{
    int range = max - min + 1;
    if (count > range)
        return NULL;

    int *unsortedlist = malloc(count * sizeof(int));
    int *used = calloc(range, sizeof(int));
    if (!used)
    {
        free(unsortedlist);
        return NULL;
    }

    for (int i = 0; i < count; i++)
    {
        int num;
        do
        {
            num = rand() % range;
        } while (used[num]);
        unsortedlist[i] = min + num;
        used[num] = 1;
    }
    free(used);
}
```

```

    return unsortedlist;
}
// Funksjon som returner en array der hverannen er 1:
int *createPatternArray(int count, int min, int max)
{
    if (count <= 0 || min > max)
        return NULL;

    int *unsorted = malloc(count * sizeof(int));
    if (!unsorted)
        return NULL;

    int range = max - min + 1;
    int duplicateValue = min; // Verdien som gjentas ofte

    for (int i = 0; i < count; i++)
    {
        if (i % 2 == 0)
        {
            // Annenhvert element får samme faste verdi
            unsorted[i] = duplicateValue;
        }
        else
        {
            // De andre elementene blir tilfeldige
            unsorted[i] = min + (rand() % range);
        }
    }

    return unsorted;
}

```

Du tenker nå sikkert, hvor er de to andre? Dette gjøres når vi lager listene i kjøretid ved å bruke bibliotekets egen sortering algoritme, `qsort`. Dette gjøres ved å definere egen "compare" funksjoner som settes inn i `qsort`; `qsort(..., comp)`. `comp` funksjonen sorter i stigenede rekkefølge (som alle de andre sorteringsalgoritmer). `reverse` reserverer en list.

```

int comp(const void *a, const void *b)
{
    int x = *(const int *)a;
    int y = *(const int *)b;

    if (x < y)
        return -1;
    if (x > y)
        return 1;
    return 0;
}
// qsort reverse
int reverse(const void *a, const void *b)
{

```

```
    return *(const int *)b - *(const int *)a;
}
```

QuickSort (fra boka)

Her brukes det tre ulike funksjoner for rekursiv calling, regning av pivotpunktet og splitting. Skal ikke forklare hvordan det fungerer siden det eksisterer en analyse i boka:

```
/*
*****
QuickSort fra boka
    t: tabell
    h: høyre
    v: venste
    m: midten
*****
*/
// Hjelpe funksjon; Median. Assisterende sort?
int median3sort(int *t, int v, int h)
{
    int m = (v + h) / 2;
    if (t[v] > t[m])
        bytt(&t[v], &t[m]);
    if (t[m] > t[h])
    {
        bytt(&t[m], &t[h]);
        if (t[v] > t[m])
            bytt(&t[v], &t[m]);
    }
    return m;
}

// Hjelpe funksjon; Splitt. BAsert på boken finner en delingsverdi
int splitt(int *t, int v, int h)
{
    int iv, ih;
    int m = median3sort(t, v, h);
    int dv = t[m];
    bytt(&t[m], &t[h - 1]);

    for (iv = v, ih = h - 1;;)
    {
        while (t[++iv] < dv)
            ;
        while (t[--ih] > dv)
            ;
        if (iv >= ih)
            break;
        bytt(&t[iv], &t[ih]);
    }
}
```

```

        bytt(&t[iv], &t[h - 1]);
        return iv;
    }

// Selve quick sort:
void quicksort(int *t, int v, int h)
{
    if (h - v > 2)
    {
        int delepos = splitt(t, v, h);
        quicksort(t, v, delepos - 1);
        quicksort(t, delepos + 1, h);
    }
    else
        median3sort(t, v, h);
}

```

Dual Pivot QuickSort (fra GeeksforGeeks)

Her brukes det to ulike funksjoner, en som har sammenlignings logikken og den andre håndterer delingen. Igjen, se nettsiden for en dypere forklaring.

```

/*
 * *****
 Quicksort fra geekforgeeks
 * *****
 */
int partition(int *arr, int low, int high, int *lp);

void DualPivotQuickSort(int *arr, int low, int high)
{
    if (low < high)
    {
        // lp means left pivot, and rp means right pivot.
        int lp, rp;
        rp = partition(arr, low, high, &lp);
        DualPivotQuickSort(arr, low, lp - 1);
        if (arr[lp] != arr[rp])
        {
            DualPivotQuickSort(arr, lp + 1, rp - 1); // If pivots are equal just
skip it
        }
        DualPivotQuickSort(arr, rp + 1, high);
    }
}

int partition(int *arr, int low, int high, int *lp)
{
    // New Pivot point to avoid worst case on identical
    int mid1 = low + (high - low) / 3;
    int mid2 = high - (high - low) / 3;

```

```
if (arr[mid1] > arr[mid2]) bytt(&arr[mid1], &arr[mid2]);
bytt(&arr[low], &arr[mid1]); // move left pivot candidate into place
bytt(&arr[high], &arr[mid2]); // move right pivot candidate into place

if (arr[low] > arr[high])
    bytt(&arr[low], &arr[high]);
// p is the left pivot, and q is the right pivot.
int j = low + 1;
int g = high - 1, k = low + 1, p = arr[low], q = arr[high]; // Better pivot
candidate fix
while (k <= g)
{
    // if elements are less than the left pivot
    if (arr[k] < p)
    {
        bytt(&arr[k], &arr[j]);
        j++;
    }

    // if elements are greater than or equal
    // to the right pivot
    else if (arr[k] >= q)
    {
        while (arr[g] > q && k < g)
            g--;
        bytt(&arr[k], &arr[g]);
        g--;
        if (arr[k] < p)
        {
            bytt(&arr[k], &arr[j]);
            j++;
        }
    }
    k++;
}
j--;
g++;

// bring pivots to their appropriate positions.
bytt(&arr[low], &arr[j]);
bytt(&arr[high], &arr[g]);

// returning the indices of the pivots.
*lp = j; // because we cannot return two elements
// from a function.

return g;
}
```

Legg merke til at denne koden er ikke helt likt fra nettsiden, fordi den har forbedringer foreslått av oppgavebeskrivelsen.

Første var velge nye pivot points, dette gjøres med å først bytte de, som gjøres her:

```
int partition(int *arr, int low, int high, int *lp)
{
    // New Pivot point to avoid worst case on identical
    int mid1 = low + (high - low) / 3;
    int mid2 = high - (high - low) / 3;

    if (arr[mid1] > arr[mid2]) bytt(&arr[mid1], &arr[mid2]);
    bytt(&arr[low], &arr[mid1]);
    bytt(&arr[high], &arr[mid2]);

    ...
}
```

Andre var å unngå duplikater, dette gjøres med en if not-setning her:

```
void DualPivotQuickSort(int *arr, int low, int high)
{
    ...
    DualPivotQuickSort(arr, low, lp - 1);
    if (arr[lp] != arr[rp])
    {
        DualPivotQuickSort(arr, lp + 1, rp - 1);
    }
    DualPivotQuickSort(arr, rp + 1, high);
    ...
}
```

Testing miljøet

Foregår i runder. En runde innebærer at begge algoritmene skal sortere samme liste type. For å unngå at den neste algoritmen sortere ikke an allerede sorterte lister brukes `memcpy`, for å skrive over og restarte lista. Eks:

```
memcpy(unsorted1, backup, count * sizeof(int)); // Reset
```

Tidstakingen skal bruke for-løkke med trials, etterligner testing i Øv2.

```
int main(){
    ...
    double total_time = 0;
    for (int i = 0; i < trials; i++)
    {
```

```

        memcpy(unsorted1, backup, count * sizeof(int)); // Reset
        clock_t start = clock();
        quicksort(unsorted1, 0, count - 1);
        clock_t end = clock();
        total_time += (double)(end - start) / CLOCKS_PER_SEC;
    }
    double avg_time = total_time / trials;

    ...
}

```

Sjekk og assert skjer etter sorteringen, etter for-loopen over.

Her kan du se en eksempel runde:

```

// RandomArray
printf("=== Round 1: Random List ===\n");
int *backup = malloc(count * sizeof(int));
memcpy(backup, unsorted1, count * sizeof(int));
assertSorted(unsorted1, count);

printf("----- Quicksort ----- \n");
printf("- Pre-sorted stats - \n");
printf("Sum: %d, \n", sum(unsorted1, count));

double total_time = 0;
for (int i = 0; i < trials; i++)
{
    memcpy(unsorted1, backup, count * sizeof(int)); // Reset
    clock_t start = clock();
    quicksort(unsorted1, 0, count - 1);
    clock_t end = clock();
    total_time += (double)(end - start) / CLOCKS_PER_SEC;
}
double avg_time = total_time / trials;
printf("- Completed - \n");
printf("Sum: %d, \n", sum(unsorted1, count));
assertSorted(unsorted1, count);
printf("[[ Tid: %.12f ]]\n", avg_time);

printf("----- Dual Pivot ----- \n");
printf("- Pre-sorted stats - \n");
printf("Sum: %d, \n", sum(unsorted1, count));

total_time = 0;
for (int i = 0; i < trials; i++)
{
    memcpy(unsorted1, backup, count * sizeof(int)); // Reset
    clock_t start = clock();
    DualPivotQuickSort(unsorted1, 0, count - 1);
    clock_t end = clock();
}

```

```

        total_time += (double)(end - start) / CLOCKS_PER_SEC;
    }
    avg_time = total_time / trials;
    printf("- Completed -\n");
    printf("Sum: %d,\n", sum(unsorted1, count));
    assertSorted(unsorted1, count);
    printf("[[ Tid: %.12f ]]\n", avg_time);
    free(unsorted1);
    printf("\n");

```

Testing

Resultat

Oppsummert er disse følgende tider:

Array \ Algoritm	QuickSort	Dual Pivot QuickSort
<i>Random List</i>	0.097933200000	0.097441800000
<i>Duplicates</i>	0.070165000000	0.046651000000
<i>Sorted</i>	0.016660800000	0.022758400000
<i>Sorted-Reverse</i>	0.027653800000	0.026135400000

Som vi ser er forskjellene større der hvor Dual Pivot QuickSort skulle være bedre i. Dual Pivot QuickSort skulle løse problemet med duplicate gjennom flere delings-punkter, som reflekteres. Men dette gjorde det vanskeligere å sortere en allerede sortert liste. Vi kan si at begge er quicksort-algoritmer og derfor har omtrent like tider i Random list, og sorted reverse kategoriene.

Her er raw terminal teksten når man kjører programmet.

```

=== Round 1: Random List ===
----- Quicksort -----
- Pre-sorted stats -
Sum: 446198416,
- Completed -
Sum: 446198416,
List is sorted
[[ Tid: 0.097933200000 ]]
----- Dual Pivot -----
- Pre-sorted stats -
Sum: 446198416,
- Completed -
Sum: 446198416,
List is sorted
[[ Tid: 0.097441800000 ]]

=== Round 2: Duplicate ===
----- Quicksort -----
- Pre-sorted stats -

```

```
Sum: -1948985820,
- Completed -
Sum: -1948985820,
List is sorted
[[ Tid: 0.070165000000 ]]
----- Dual Pivot -----
- Pre-sorted stats -
Sum: -1948985820,
- Completed -
Sum: -1948985820,
List is sorted
[[ Tid: 0.046651000000 ]]

=== Round 3: Already Sorted ===
List is sorted
----- Quicksort -----
- Pre-sorted stats -
Sum: 446198416,
- Completed -
Sum: 446198416,
List is sorted
[[ Tid: 0.016660800000 ]]
----- Dual Pivot -----
- Pre-sorted stats -
Sum: 446198416,
- Completed -
Sum: 446198416,
List is sorted
[[ Tid: 0.022758400000 ]]

=== Round 4: Reverse Sorted (already sorted but reverse order) ===
----- Quicksort -----
- Pre-sorted stats -
Sum: 446198416,
- Completed -
Sum: 446198416,
List is sorted
[[ Tid: 0.027653800000 ]]
----- Dual Pivot -----
- Pre-sorted stats -
Sum: 446198416,
- Completed -
Sum: 446198416,
List is sorted
[[ Tid: 0.026135400000 ]]
```